

Chapter 5

JWT Authentication



From the problem of stateless identity to a working `JwtFilter` — how FoodBridge recognizes an authenticated user on every request.

Part 1 — The Need for JWT Authentication

5.1 INTRODUCTION

In the previous chapter, we successfully implemented user login. When a user entered a valid email and password, our backend generated a JSON Web Token (JWT) and returned it to the client. At first, this may seem like the authentication process is complete. However, a new question immediately arises:

How will the backend know that the next request is coming from the same authenticated user?

Suppose Rahul logs in successfully and receives a JWT. A few minutes later, he sends another request to create a food donation. Should the backend ask for his credentials again? Store his login information in memory? Or is there a better solution? This chapter answers these questions by introducing JWT Authentication.

5.2 THE PROBLEM AFTER LOGIN

During login, the backend verifies the user's email and password, then generates a JWT and returns it to the client. Now imagine the client wants to access another API, such as `POST /donations`. Unlike the login request, this request does not contain the user's email or password.

How does the backend know who sent this request?

Without some way of identifying the user, every protected API would be insecure.

5.3 A NAIVE SOLUTION

One possible solution is to ask the user for their email and password every time they send a request. Although this works, it creates a poor user experience and increases security risk, since passwords would travel across the network far more often than necessary. Clearly, this is not an ideal solution.

5.4 SESSION-BASED AUTHENTICATION

Before JWT became popular, most web applications used **Session Authentication**:

- User logs in
- Server verifies credentials
- Server creates a session and stores it in memory
- Server sends a Session ID to the browser
- Browser sends the Session ID with every future request

The server checks whether that Session ID exists; if it does, the request is considered authenticated.

5.5 PROBLEMS WITH SESSIONS

1 Server Memory

Every logged-in user requires the server to store session data. If one million users are logged in, the server must maintain one million sessions — increasing memory usage significantly.

2 Scalability

If Rahul logs into Server A, his session is stored only there. If his next request is routed to Server B by a load balancer, that server has no knowledge of his session, and he appears to be logged out. This problem worsens as the number of servers increases.

3 Session Synchronization

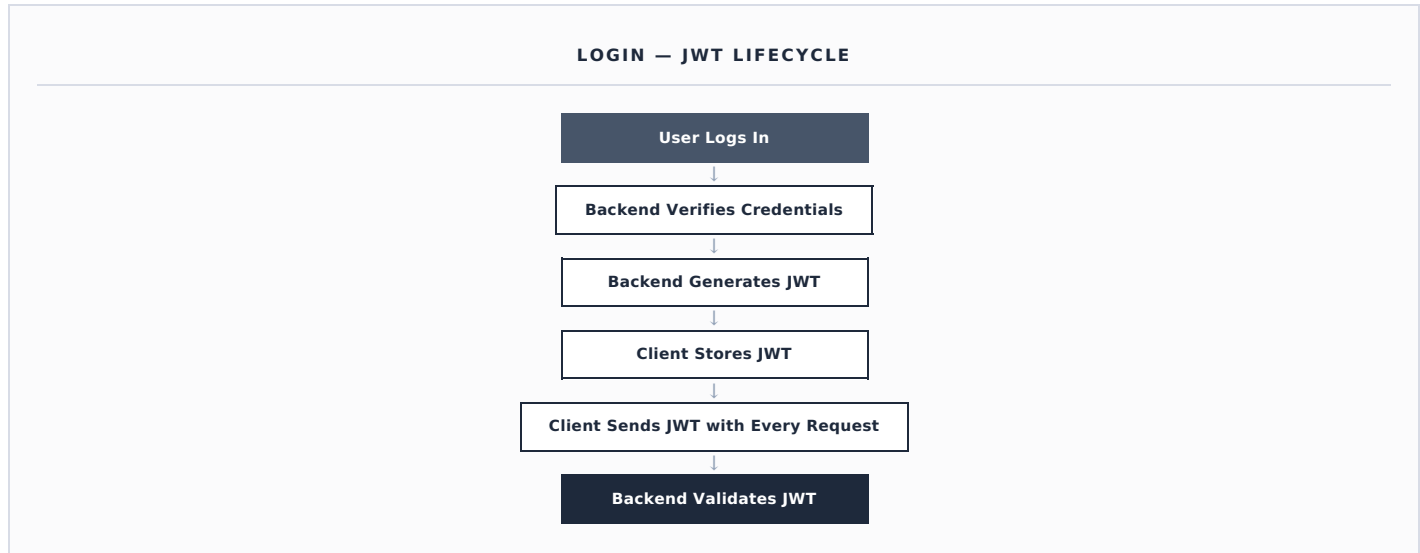
Large applications often solve this by storing sessions in Redis or another shared storage system. Although effective, this increases the complexity of the system.

5.6 STATELESS AUTHENTICATION

Modern REST APIs usually avoid server-side sessions. Instead, they use **Stateless Authentication**. The word *stateless* means the server does not remember previous requests — each request must contain all the information needed for authentication. Instead of the server keeping login information in memory, the client carries proof of authentication. This proof is the JWT.

5.7 WHAT IS JWT?

JWT stands for **JSON Web Token** — a digitally signed token that contains information about an authenticated user. Instead of storing authentication information on the server, the server stores it inside the token.



Notice something important: the backend does **not** remember the user. Instead, the JWT proves that the user has already been authenticated.

5.8 WHERE IS THE JWT STORED?

After login, the client stores the JWT — in Browser Local Storage, Session Storage, Secure HTTP Cookies, or Mobile Application Storage. The backend does not care where it is stored; its only responsibility is to validate the token whenever it is received.

5.9 SENDING THE JWT

Every protected request includes the JWT inside the HTTP **Authorization** header:

```
GET /donations

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLT...
```

The word **Bearer** tells the backend that the following value is an authentication token. Our **JwtFilter** will later read this header before the request reaches the controller.

5.10 WHY USE THE AUTHORIZATION HEADER?

HTTP requests can contain many headers, each with a different purpose. The `Authorization` header is the standard location for sending authentication credentials. By placing the JWT here, clients and servers follow a widely accepted HTTP convention, and frameworks such as Spring Security integrate naturally with token-based authentication.

5.11 SESSION AUTHENTICATION VS JWT AUTHENTICATION

SESSION AUTHENTICATION	JWT AUTHENTICATION
Server stores session	Client stores JWT
Requires server memory	No server memory for sessions
Uses Session ID	Uses JWT
Server remembers users	Server validates tokens
Harder to scale	Easier to scale
Stateful	Stateless

5.12 KEY TAKEAWAYS

- Logging in once is not enough — the backend must identify users on every future request
- Asking users for their password on every request is impractical and insecure
- Session Authentication stores user information on the server
- JWT Authentication stores authentication information inside a signed token
- REST APIs commonly use JWT because it supports stateless communication and scales well
- Every protected request includes the JWT inside the `Authorization` header

In the next section, we will explore the internal structure of a JWT and understand how the backend generates, signs, and validates these tokens.

Part 2 — Understanding JWT Internals

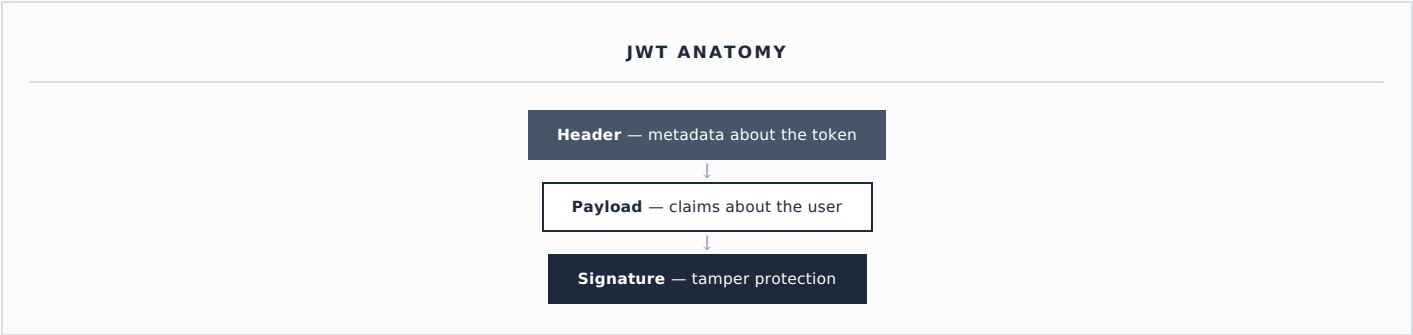
5.13 WHAT EXACTLY IS A JWT?

Many beginners think a JWT is just a random encrypted string. It is not. A JWT is a structured token made up of three parts — **Header**, **Payload**, and **Signature** — joined together with periods:

```
Header.Payload.Signature
```

5.14 STRUCTURE OF A JWT

Think of a passport: it contains information about the issuing country, information about the person, and a security feature proving it hasn't been forged. A JWT works similarly — the Header describes the token, the Payload contains information about the user, and the Signature protects the token from being modified.



5.15 THE HEADER

The Header contains metadata about the JWT — information about the token itself:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

`alg` specifies which algorithm signed the token — in our project, **HS256** (HMAC SHA-256), used together with our secret key. `typ` tells software that the value received is a JSON Web Token.

5.16 THE PAYLOAD

The Payload contains information about the authenticated user, stored as **Claims**:

```
{
  "sub":"rahul@gmail.com",
  "role":"DONOR",
  "iat":1712345678,
  "exp":1712349278
}
```

The payload answers: who logged in, when was the token created, when does it expire, and what role does the user have?

5.17 -5.19 CLAIMS — STANDARD AND CUSTOM

Claims are key-value pairs inside the payload, divided into two categories: **Standard Claims**, predefined by the JWT specification and understood by almost every JWT library, and **Custom Claims**, added by the application itself.

CLAIM	MEANING IN FOODBRIDGE
<code>sub</code> — Subject	Identifies the owner of the token (the user's email). Read via <code>extractEmail(jwt)</code> .
<code>iat</code> — Issued At	The time the token was created; helps identify when authentication occurred.
<code>exp</code> — Expiration	The expiry time. Once passed, the token is rejected via <code>isTokenExpired(jwt)</code> .
<code>role</code> — Custom Claim	Not part of the JWT spec; added by us so Spring Security can later determine access permissions.

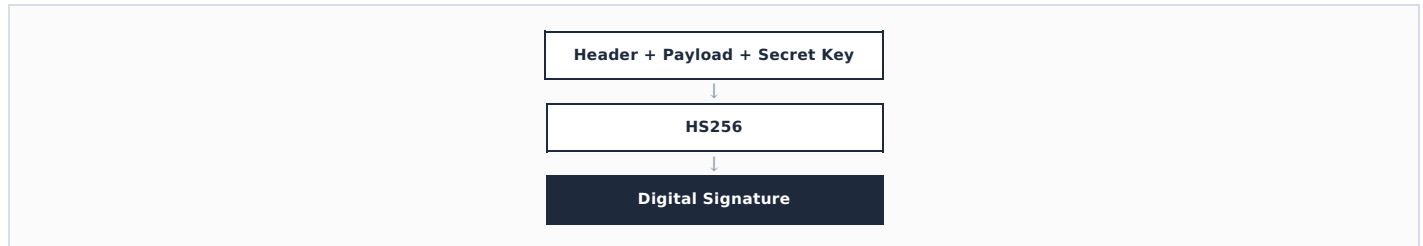
5.20 HEADER + PAYLOAD ARE NOT SECRET

Many beginners think a JWT is encrypted. It is not — the Header and Payload are only **Base64URL encoded**. Encoding is not encryption; anyone who receives the token can decode these sections and read their contents.

Never store sensitive information inside a JWT — passwords, credit card numbers, OTPs, or secret API keys. In FoodBridge we only store safe information: email, role, issue time, and expiration time.

5.21 THE SIGNATURE

The Signature is the most important part of a JWT. Its purpose is **not** to hide information — it proves that the token has not been modified. Spring creates it from the Header, Payload, Secret Key, and the HS256 algorithm:



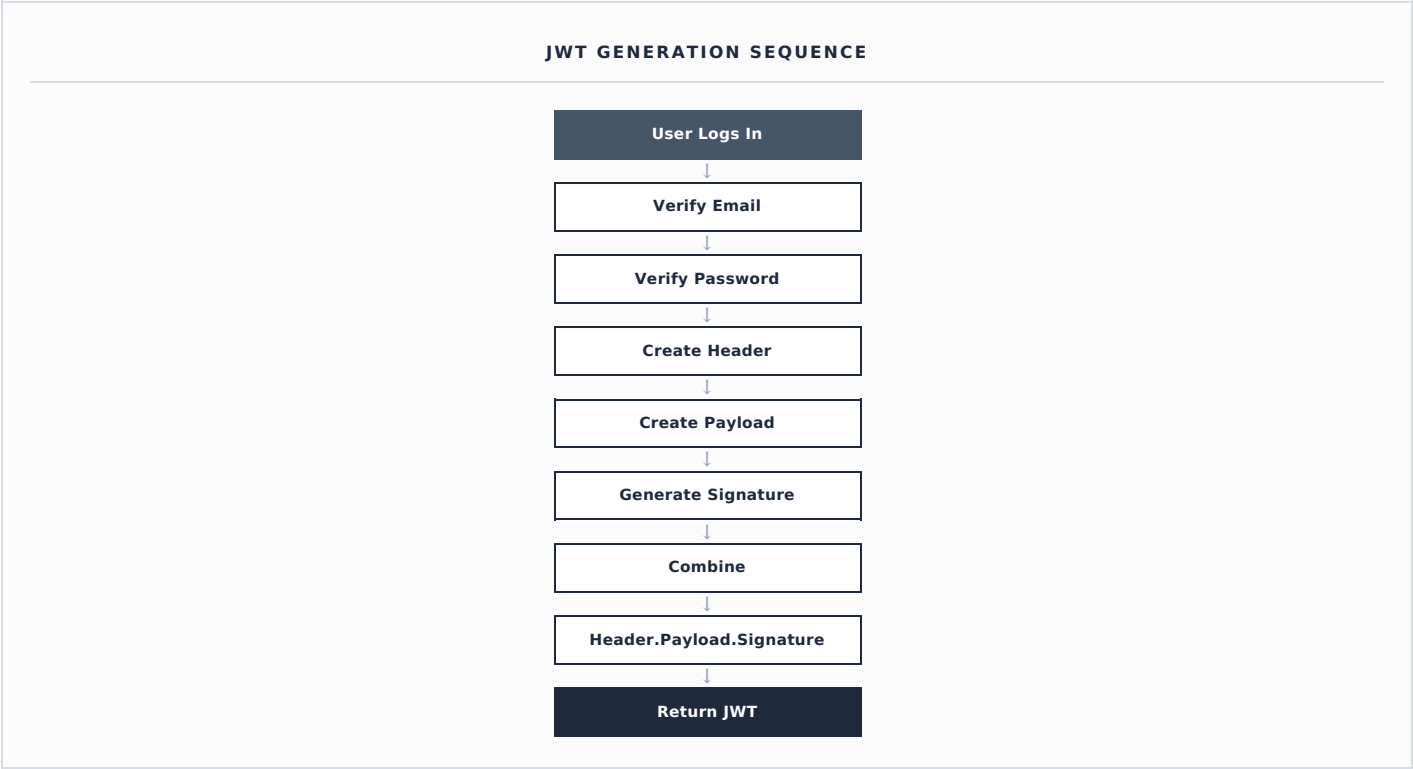
5.22 WHY DO WE NEED THE SIGNATURE?

Suppose a user edits their payload from `"role":"DONOR"` to `"role":"ADMIN"`. Without a signature, the server would trust the modified token, letting anyone become an administrator. Whenever the payload changes — even by a single character — the signature becomes invalid, and our backend immediately rejects the token.

5.23 SECRET KEY

The Secret Key is known only to the backend and never shared with clients. When generating a JWT, Spring combines the Header, Payload, and Secret Key to create the signature. When validating a JWT, Spring performs the same calculation again — if both signatures match, the token is valid; if they differ, someone has modified it.

5.24 COMPLETE JWT GENERATION PROCESS



5.25 CONCEPTS LEARNED

JWT Structure	Header	Payload	Signature	Claims	Standard Claims	Custom Claims	Subject (sub)	Issued At (iat)
Expiration (exp)	Role Claim	Secret Key	HS256	Base64URL Encoding	Digital Signature			

CHAPTER SUMMARY — PART 2

A JWT is not a random string but a structured token consisting of a Header, Payload, and Signature. The Header describes how the token is signed, the Payload stores claims about the authenticated user, and the Signature protects the token from tampering.

Standard claims such as `sub`, `iat`, and `exp` provide common information, while custom claims like `role` allow applications to include additional data. Because the Header and Payload are only encoded — not encrypted — sensitive information should never be stored inside them.

Part 3 — Implementing JWT Authentication (JwtService)

5.26 FROM THEORY TO IMPLEMENTATION

Knowing the theory is not enough. Our application must be able to perform five practical tasks: generate a JWT after login, extract information from an existing JWT, check whether it has expired, verify that it belongs to the correct user, and reject invalid or tampered JWTs. This logic lives in a dedicated class called `JwtService`.

5.27 WHY A SEPARATE JWTSERVICE?

Writing JWT code directly inside `UserService` would mix two different responsibilities. `UserService` handles user-related business logic, while JWT creation and validation are authentication-specific tasks. This follows the **Single Responsibility Principle (SRP)**:

CLASS	RESPONSIBILITY
UserService	Handles user authentication and business rules
JwtService	Handles everything related to JWTs

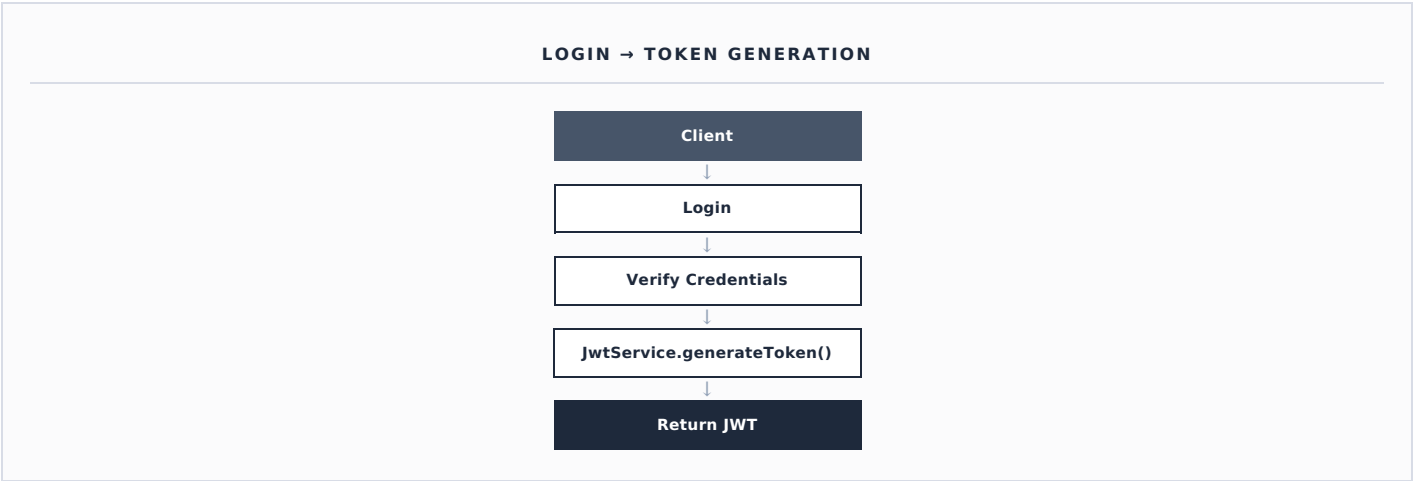
5.28 RESPONSIBILITIES OF JWTSERVICE

- Generate a JWT
- Read claims from a JWT
- Extract the user's email
- Read the expiration time
- Check whether the token has expired
- Validate that a token belongs to the expected user

5.29-5.30 GENERATING A JWT AND BUILDING THE PAYLOAD

After a successful login, `UserService` calls:

```
String token = jwtService.generateToken(user);
```



The payload includes the Subject (`sub` — user's email), Role (`role` — DONOR, NGO, or ADMIN), Issued At (`iat`), and Expiration (`exp`):

```
{
  "sub": "rahul@example.com",
  "role": "DONOR",
  "iat": "...",
  "exp": "..."
}
```

5.31-5.32 WHY STORE THE ROLE?

If the token only stored the email, Spring Security would know *who* sent the request but not *what permissions* they have. Storing the role prepares the application for role-based authorization. Since `Role` is a Java enum, we convert it to text using:

```
.claim("role", user.getRole().name())
```

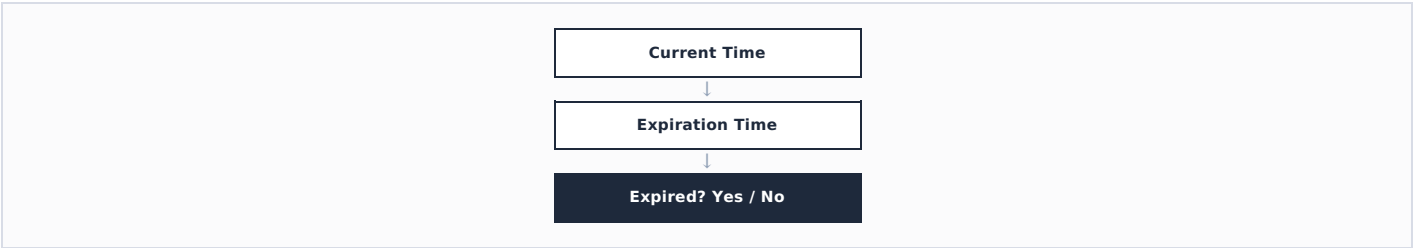
5.33-5.34 EXTRACTING THE EMAIL

Whenever a protected request arrives, the backend must read information from the JWT. The most important method is:

```
extractEmail(jwt)
```

Internally, this reads the `sub` claim. If the payload contains `"sub": "rahul@example.com"`, then `extractEmail(jwt)` returns `rahul@example.com`, allowing the application to identify the owner of the token.

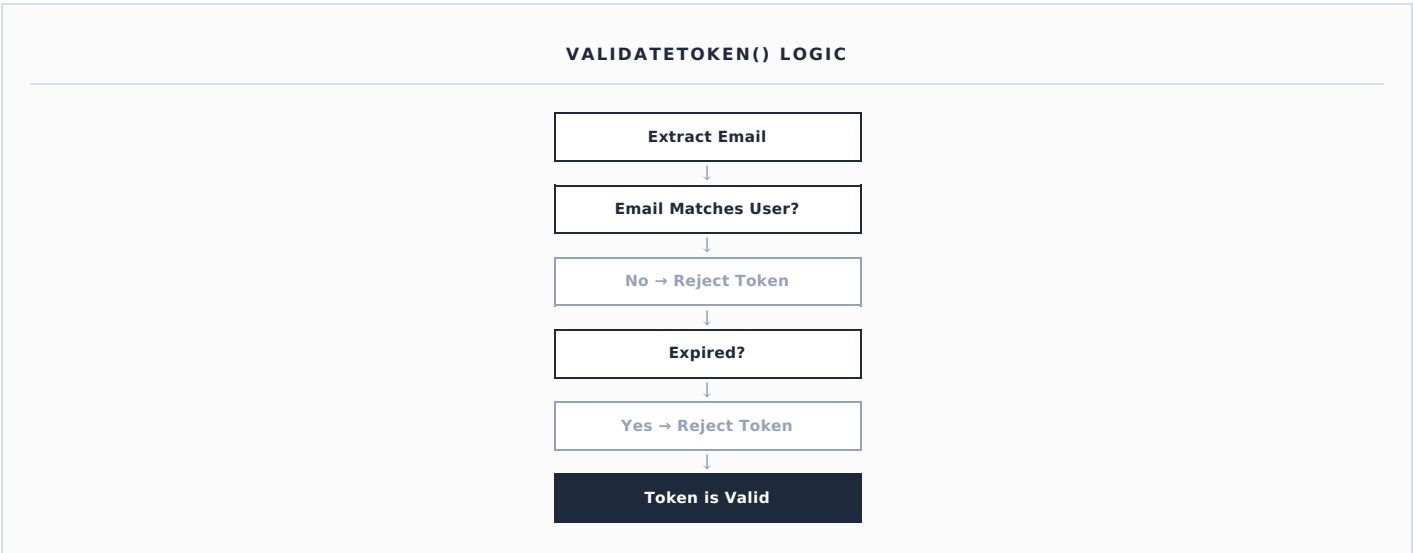
5.35-5.36 CHECKING EXPIRATION



If a stolen JWT never expired, an attacker could use it indefinitely. Adding an expiration time reduces this risk — even a compromised token eventually becomes useless.

5.37-5.39 VALIDATING THE JWT

Validation consists of two checks: does the email inside the token match the expected user, and has the token expired? Only if both conditions are true is the token considered valid.



Comparing the email ensures the token belongs to the expected user — without this check, someone could use Rahul's token while claiming to be Priya.

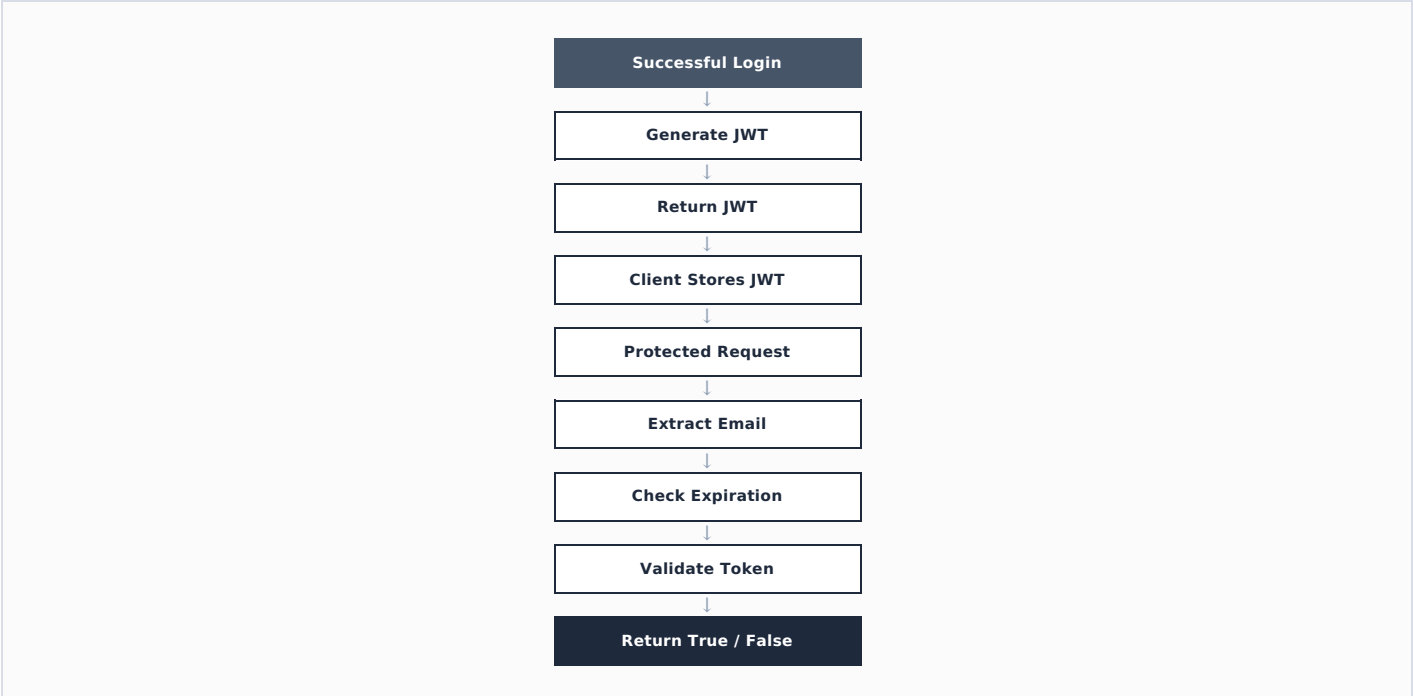
5.40 WHY USE EQUALS()?

In Java, two different `String` objects may contain the same text. Using `email == user.getEmail()` compares object references, not their contents. Instead, we use:

```
email.equals(user.getEmail());
```

This compares the actual text — an important correction made during development.

5.41 COMPLETE JWTSERVICE WORKFLOW



5.42 CONCEPTS LEARNED

Dedicated JwtService	Token Generation	Claim Building	Role Claim	extractEmail()	Token Expiration	Token Validation
String.equals()						

CHAPTER SUMMARY — PART 3

`JwtService` acts as the central component responsible for all JWT-related operations in FoodBridge. It creates tokens after successful login, embeds important claims such as the user's email and role, extracts information from incoming tokens, checks whether they have expired, and validates that they belong to the correct user — keeping authentication concerns isolated from the rest of the application.

Part 4 — Authenticating Protected Requests

5.43 INTRODUCTION

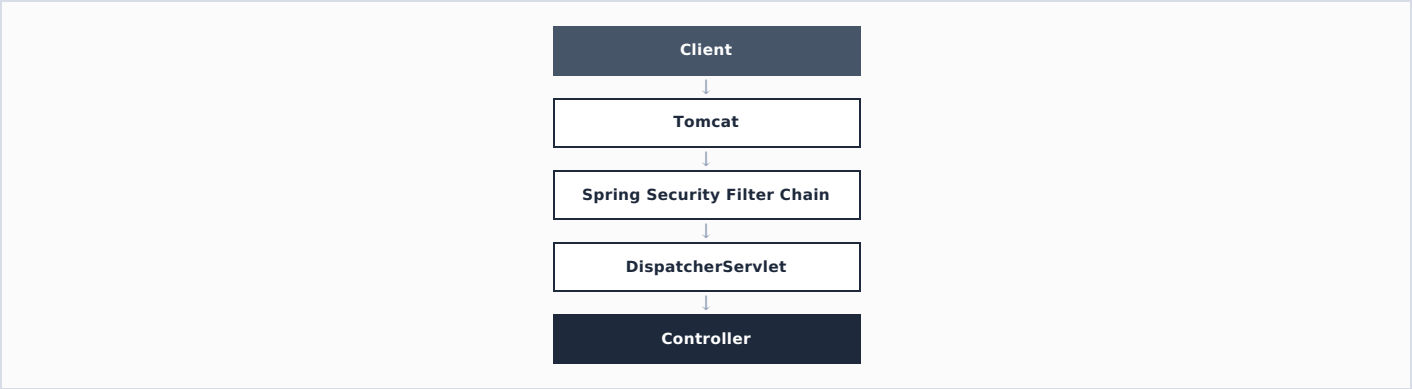
We've learned how our application generates and validates JWTs using `JwtService`. But one question remains:

Who calls `JwtService.validateToken()`?

A client simply sends an HTTP request with a JWT in the `Authorization` header. Neither the controller nor the service layer validates the token. Some component must intercept the request **before** it reaches the controller — this responsibility belongs to our custom `JwtFilter`.

5.44-5.46 WHY A FILTER, AND WHY EVERY REQUEST PASSES THROUGH IT

If a protected request went directly to the controller, anyone could call protected APIs by simply knowing the URL. Instead, every request must pass through a security checkpoint — the Spring Security Filter Chain, inserted between Tomcat and Spring MVC:



Even public endpoints such as `/users/login` and `/users` pass through this chain — it does not automatically reject every request. Instead, it applies the rules configured in `SecurityConfig`: `.requestMatchers("/users/**").permitAll()` for public endpoints, and `.anyRequest().authenticated()` for everything else.

5.47 OUR CUSTOM JWTFILTER

Spring Security cannot understand our JWT automatically — it doesn't know how to read it, which secret key we use, which claims we store, or how to validate it. Our `JwtFilter` exists to:

- Read the Authorization header
- Extract the JWT
- Extract the email
- Load the user
- Validate the token
- Create an Authentication object
- Store it inside the Security Context

Only after these steps can Spring Security treat the request as authenticated. Our filter extends `OncePerRequestFilter`, which guarantees it executes exactly once per HTTP request.

The Eight-Step JwtFilter Sequence

1 Read the Authorization Header

```
String authHeader = request.getHeader("Authorization");
```

If the header is missing or does not begin with `"Bearer "`, there is no JWT to validate, and the filter simply forwards the request — which is why login and registration continue to work without a token.

2 Extract the JWT

If the header looks like `Authorization: Bearer eyJhbGc...`, we remove the `"Bearer "` prefix. The remaining string is the actual JWT, passed to `JwtService`.

3 Extract the Email

```
jwtService.extractEmail(jwt);
```

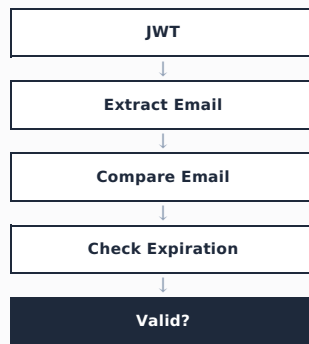
This reads the `sub` claim, allowing the application to identify which user owns the token.

4 Load the User

```
userService.getUserByEmail(email);
```

Knowing the email is not enough — we still need the corresponding `User` object from the database.

5 Validate the JWT



If validation fails, the request is rejected. If it succeeds, the request continues.

6 Create the Authentication Object

```
UsernamePasswordAuthenticationToken authentication =  
    new UsernamePasswordAuthenticationToken(user, null, null);
```

This line does **not** authenticate the user — the user has already been authenticated because we successfully validated the JWT. It simply creates an object representing that authenticated user, carrying that identity through the rest of the request.

7 SecurityContextHolder

```
SecurityContextHolder  
    .getContext()  
    .setAuthentication(authentication);
```

This tells Spring Security “this request belongs to an authenticated user.” From this point, every component in the current request can retrieve the authenticated user from the `SecurityContext`. It does not authenticate anyone — it simply stores the authentication information for the current request.

8 Continue the Filter Chain

```
filterChain.doFilter(request, response);
```

This forwards the request to the next filter. Eventually it reaches the `DispatcherServlet`, which already knows the user has been authenticated.

SecurityConfig

Our `SecurityConfig` defines which requests require authentication:

```
http
.csrf(csrf -> csrf.disable())
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/users/**").permitAll()
    .anyRequest().authenticated()
)
.addFilterBefore(jwtFilter,
    UsernamePasswordAuthenticationFilter.class);
```

Three important things happen here: CSRF protection is disabled because our application uses JWT instead of session-based authentication; login and registration are publicly accessible; and our custom `JwtFilter` is inserted into the filter chain before Spring's default username/password authentication filter.

Complete Protected Request Flow



Authentication vs Authorization

AUTHENTICATION	AUTHORIZATION
“Who are you?”	“What are you allowed to do?”
Solved by JWT, JwtFilter, SecurityContext	Depends on roles and permissions
Implemented in this chapter	Implemented in the next chapter

COMMON MISTAKES

- 1

Thinking UsernamePasswordAuthenticationToken Authenticates the User

It does not — it only represents an already authenticated user.
- 2

Thinking SecurityContextHolder Validates the JWT

Validation happens in `JwtService`, not in the context holder.
- 3

Forgetting to Call filterChain.doFilter()

Without this call, the request stops inside the filter and never reaches the controller.
- 4

Believing Public Endpoints Bypass the Filter Chain

Every request passes through the filter chain — public endpoints are simply permitted by the configured authorization rules.

CHAPTER SUMMARY

In this chapter, we completed the JWT authentication mechanism for FoodBridge. Every protected request first passes through the Spring Security Filter Chain, where our custom `JwtFilter` intercepts it before it reaches the controller. The filter extracts the JWT from the `Authorization` header, identifies the user by reading the `sub` claim, loads the corresponding user from the database, and validates the token using `JwtService`.

If the token is valid, an `Authentication` object is created and stored inside the `SecurityContext`, allowing Spring Security to recognize the request as authenticated. With authentication complete, the application is now ready to implement **authorization**, where user roles determine which actions each authenticated user is allowed to perform.